

Synchronizer Service

Overview

The Synchronizer Service is responsible for processing listing-related events and synchronizing data into MongoDB. It acts as a consumer in an event-driven architecture, ensuring that marketplace data is stored and updated reliably.

Architecture

The service follows an event-driven design:

1. Messages are received from RabbitMQ
 2. Messages are wrapped in an **event envelope**
 3. The payload is deserialized into domain models
 4. Data is transformed into a MongoDB-friendly format
 5. Listings are upserted or deleted
 6. Processed events are stored to ensure idempotency
-

Event Envelope

All messages follow this structure:

```
{
  "eventId": "uuid",
  "eventType": "ListingCreated",
  "payload": {
    "guid": "123",
    "personGUID": "abc",
    "listingDetails": {
      "title": "Chair",
      "price": 100
    }
  }
}
```

Features

Idempotency

- Prevents duplicate processing
- Stores processed events in MySQL

Event Handling

Supports:

- ListingCreated
 - ListingUpdated
 - ListingDeleted
-

MongoDB Integration

- Stores listings as SalesPost
 - Uses Upsert for create/update
 - Deletes based on GUID
-

Data Transformation

Transforms:

- IncomingListing → SalesPost

Handles:

- Colors
 - Categories
 - Images
-

Docker Setup

Runs alongside:

- RabbitMQ
- MongoDB
- Other microservices

Start system:

```
docker compose up --build
```

Environment Variables

```
MONGO_CONNECTION=your_mongo_connection  
MYSQL_CONNECTION=server=mysql;database=emaildb;user=root;password=1234
```

```
RABBITMQ_HOST=rabbitmq
QUEUE_NAME=listing_queue
```

Flow Summary

RabbitMQ → Worker → Envelope → Transform → MongoDB → Save Event

Design Decisions

Event-Driven Architecture

- Loose coupling between services
- Scalable and asynchronous

Envelope Pattern

- Standardizes communication
- Enables tracking

Idempotency

- Prevents duplicate DB operations
 - Critical in distributed systems
-

Future Improvements

- Add retry handling
 - Implement DLQ
 - Add validation layer
 - Improve logging
-

Notes

This service complements the Email Service by ensuring that listing data is consistently stored and updated across the system.
